

Agentic AI Foundations



Hardik Gupta

Learning Objectives Covered During the Sunday Live Class

- Agent Types & Intelligence Levels
 - Reflex agents (reactive, condition-action rules) → Goal-based agents (planning with search) → Utility agents (optimizing outcomes)
- LLM Fundamentals & Limitations
 - Transformer architecture & attention mechanisms
 - Core constraints: static knowledge, no memory, passive output, hallucination risks
- LLM-Based Agent Architecture
 - ReAct loop (Reason → Act → Observe)
 - Four pillars: Brain (LLM), Tools, Memory (short/long-term), Planning (CoT, reflection)

Learning Objectives Covered During the Sunday Live Class

- Core Agentic Design Patterns
 - **Routing:** Dynamic decision-making for next actions/tools
 - **Reflection:** Self-critique to refine outputs before finalizing
- Real-World SWE Applications
 - Built CSV FAQ Agent (natural language queries over structured data)
 - Designed CRM Lead Qualifier Agent with LLM-based routing logic

What are we going to do today

To ensure learners can effectively apply the learning objectives from the Sunday live class:

- We will solve MCQs and assignment problems to reinforce core concepts.
- For the assignment released after the live session, the instructor will walk through the complete solution and explain the final implementation approach in detail.

Let's Unpack the MCQs

MCQ: 1

What is the fundamental mechanism behind a Large Language Model's text generation?

- A** Predicting the next token based on statistical probability.
- B** Retrieving pre-written answers from a structured database.
- C** Executing Python code to logically derive truth.
- D** Simulating a human brain's biological neurons.

MCQ: 1

What is the fundamental mechanism behind a Large Language Model's text generation?

- A** Predicting the next token based on statistical probability.
- B** Retrieving pre-written answers from a structured database.
- C** Executing Python code to logically derive truth.
- D** Simulating a human brain's biological neurons.

Rationale: LLMs are probabilistic engines trained to predict the most likely next word (token) given the previous context.

MCQ: 2

What is a "Hallucination" in the context of LLMs?

A

When the model crashes due to overload.

B

When the model confidently presents incorrect information as fact.

C

When the model generates text in the wrong language.

D

When the model refuses to answer a sensitive question.

MCQ: 2

What is a "Hallucination" in the context of LLMs?

- A

 When the model crashes due to overload.
- B

 When the model confidently presents incorrect information as fact.
- C

 When the model generates text in the wrong language.
- D

 When the model refuses to answer a sensitive question.

Rationale: Because LLMs prioritize fluent text generation, they can convincingly generate false statements that look plausible, known as hallucinations.

MCQ: 3

What primary addition allows a standard LLM to function as an "Agent"?

A

Access to external tools and a reasoning loop (e.g., ReAct).

B

More training data and parameters

C

A physical robot body.

D

Faster processing speed.

MCQ: 3

What primary addition allows a standard LLM to function as an "Agent"?

- A** Access to external tools and a reasoning loop (e.g., ReAct).
- B** More training data and parameters
- C** A physical robot body.
- D** Faster processing speed.

Rationale: To be an agent, the model must be able to act on the world (tools) and plan its actions (reasoning/memory). Without these, it is just a passive text generator.

MCQ: 4

According to the foundational formula for AI agents, which three components combined with an LLM create an agent?

- A** Data, Algorithms, and Models.
- B** Input, Processing, and Output.
- C** Prompt, Tools, and Memory.
- D** Sensors, Controllers, and Actuators.

MCQ: 4

According to the foundational formula for AI agents, which three components combined with an LLM create an agent?

A

Data, Algorithms, and Models.

B

Input, Processing, and Output.

C

Prompt, Tools, and Memory.

D

Sensors, Controllers, and Actuators.

Rationale: The foundational formula for an AI agent is: Agent = (Prompt + Tools + Memory) × LLM. Prompt defines the agent's behavior, Tools enable actions (APIs, search, code execution), and Memory allows the agent to remember context between turns.

MCQ: 5

What are the three stages that compose the core agent loop for problem-solving?

A Plan, Execute, Review

B Think, Act, Observe

C Input, Process, Output

D Read, Decide, Respond

MCQ: 5

What are the three stages that compose the core agent loop for problem-solving?

A Plan, Execute, Review

B Think, Act, Observe

C Input, Process, Output

D Read, Decide, Respond

Rationale: The core Agent Loop follows the cycle: Think (analyze the request) → Act (decide which tool to call or action to take) → Observe (read the tool's output or outcome). This loop repeats until the agent reaches a final answer or completes the task.

MCQ: 6

In the context of modern AI agents, what does the ReAct framework stand for?

A

Reacting and Acting

B

Reading and Writing

C

Reasoning and Acting

D

Retrieval and Augmentation

MCQ: 6

In the context of modern AI agents, what does the ReAct framework stand for?

A

Reacting and Acting

B

Reading and Writing

C

Reasoning and Acting

D

Retrieval and Augmentation

Rationale: The ReAct framework stands for Reasoning + Acting. It describes the standard loop for modern agents: Thought (analyze request) → Action (select and call a tool) → Observation (process the tool's output) → Repeat, continuing until the task is resolved.

MCQ: 7

How is long-term memory typically implemented for AI agents to enable persistent knowledge recall?

A

As the model's context window, limited by token capacity.

B

Through vector databases that store information for semantic retrieval.

C

Via the model's internal parameter weights.

D

Using temporary session storage cleared after each interaction

MCQ: 7

How is long-term memory typically implemented for AI agents to enable persistent knowledge recall?

A

As the model's context window, limited by token capacity.

B

Through vector databases that store information for semantic retrieval.

C

Via the model's internal parameter weights.

D

Using temporary session storage cleared after each interaction

Rationale: Long-term memory for agents is implemented using vector databases (RAG) that act as the agent's "hard drive," storing vast amounts of information. The agent retrieves relevant memories based on semantic similarity to the current task, enabling persistent knowledge beyond a single session's context window.

MCQ: 8

What distinguishes an autonomous agent from a predefined workflow in terms of orchestration?

A

Autonomous agents use code-defined fixed pipelines, while workflows use dynamic LLM control.

B

Autonomous agents rely on human manual input, while workflows are fully automated.

C

Autonomous agents use dynamic LLM-based decision making, while workflows use code-defined fixed pipelines.

D

Both use identical orchestration methods with no meaningful distinction.

MCQ: 8

What distinguishes an autonomous agent from a predefined workflow in terms of orchestration?

A

Autonomous agents use code-defined fixed pipelines, while workflows use dynamic LLM control.

B

Autonomous agents rely on human manual input, while workflows are fully automated.

C

Autonomous agents use dynamic LLM-based decision making, while workflows use code-defined fixed pipelines.

D

Both use identical orchestration methods with no meaningful distinction.

Rationale: Autonomous agents use dynamic, LLM-driven orchestration that adapts decisions on the fly, offering high flexibility but lower predictability. In contrast, predefined workflows use code-defined fixed pipelines with known steps in advance, offering higher predictability but lower adaptability.

MCQ: 9

What primary capability do tools provide to an LLM-based agent system?

A

Enhancing the creativity and fluency of generated text.

B

Reducing computational costs and inference time.

C

Transforming text generation into the ability to perform actions in external systems.

D

Encrypting user data for improved security compliance.

MCQ: 9

What primary capability do tools provide to an LLM-based agent system?

A

Enhancing the creativity and fluency of generated text.

B

Reducing computational costs and inference time.

C

Transforming text generation into the ability to perform actions in external systems.

D

Encrypting user data for improved security compliance.

Rationale: Tools enable LLMs to move beyond text generation by allowing them to retrieve data from APIs or databases, execute actions in external systems (e.g., sending emails, calling web services), and run deterministic logic—effectively turning passive text generators into systems that can act in the world.

MCQ: 10

Which planning technique involves an agent explicitly stepping through its logic to break down complex problems?

A

Tree of Thoughts

B

Reflection

C

Chain of Thoughts

D

Semantic Retrieval

MCQ: 10

Which planning technique involves an agent explicitly stepping through its logic to break down complex problems?

A

Tree of Thoughts

B

Reflection

C

Chain of Thoughts

D

Semantic Retrieval

Rationale: Chain of Thought is a planning technique where agents explicitly step through their reasoning logic to decompose complex goals into manageable sub-tasks. This improves reliability by making the agent's problem-solving process transparent and structured, as opposed to attempting complex tasks in a single step.

Now that we've unpacked the MCQs and reinforced core concepts—let's dive into the hands-on assignment where theory meets practice.

Autonomous IT Support Agent

Autonomous IT Support Agent

- **Objective:** Build an AI agent that acts as the "first responder" for server incidents. It must:
 - **Investigate:** Check server health and logs when a user reports an issue.
 - **Act:** If CPU is critical (>90%), it should **Restart** the service.
 - **Escalate:** If the issue is complex or logs show a major problem it should **Escalate** to a human.
- **Your Task:** complete the code given by filling in the sections marked ### TODO.

Assignment

Specific Tasks to Complete

1. Implement the Restart Tool

- Define the `restart_service` function
- Print a message saying `"-> TOOL: Restarting service..."`
- Return a JSON string confirming the restart was successful.

2. Implement the Escalation Tool

- Define the `escalate_to_engineer` function
- Print a message saying `"-> TOOL: Escalating to human..."`
- Return a JSON string confirming the ticket was created.

Specific Tasks to Complete

3. Define schema for restart_service:

- Write a description for the AI
- Define the 'server_id' parameter

4. Define Schema for escalate_to_engineer

- Define the 'summary' parameter (type string)

5. Append the result to messages

**Let's now look at the
Solution**

Solution Overview

The assignment requires implementing two critical tools for an AI agent that acts as a Level 1 IT Incident Responder:

1. `restart_service(server_id: str) -> str`

- Triggered when CPU usage exceeds 90% (critical threshold)
- Must simulate restarting a service and return confirmation in JSON format
- Should print a clear diagnostic message showing action taken

2. `escalate_to_engineer(summary: str) -> str`

- Triggered when logs indicate complex issues (e.g., dependency failures, payment gateway errors)
- Must simulate creating a human escalation ticket with the provided summary
- Should print an escalation diagnostic message

TASK 1: To Implement the Restart Tool

- Create a function that simulates restarting a server service

```
def restart_service(server_id: str) -> str:
    """Simulates a service restart."""
    print(f"-> TOOL: Restarting service {server_id}...")

    # In a real scenario, this would run a subprocess command or API call
    result = {
        "server_id": server_id,
        "status": "success",
        "message": "Service restart command issued successfully."
    }
    return json.dumps(result)
```

- Prints execution message for observability and returns JSON confirmation of successful restart. Simulates real-world service restart command.

TASK 2: To Implement the Escalation Tool

- Create a function that escalates complex issues to human engineers when automated fixes would be ineffective.

```
def escalate_to_engineer(summary: str) -> str:
    """Simulates sending an alert to a human."""
    print(f"-> TOOL: Escalating to human engineer. Reason: {summary}")

    # In a real scenario, this would send a Slack message or PagerDuty alert
    result = {
        "status": "escalated",
        "ticket_id": "INC-999",
        "assigned_to": "On-Call Engineer"
    }
    return json.dumps(result)
```

- Prints escalation notification with reason summary. Returns JSON with unique ticket ID for tracking human-engineer handoff.

TASK 3: To Define Schema for restart_service

- Create OpenAI function schema that enables the LLM to understand when and how to use the restart tool.

```
{
  "type": "function",
  "function": {
    "name": "restart_service",
    "description": "Restarts a specific server service. Use this when CPU usage is critically high or the process is unresponsive.",
    "parameters": {
      "type": "object",
      "properties": {
        "server_id": {"type": "string", "description": "The ID of the server to restart."}
      },
      "required": ["server_id"]
    }
  }
},
```

- OpenAI function schema defining when to use restart (high CPU/unresponsive) and required parameter (server_id) for proper LLM tool invocation.

TASK 4: To Define Schema for escalate_to_engineer

- Create OpenAI function schema that enables the LLM to understand when escalation is appropriate.

```
{
  "type": "function",
  "function": {
    "name": "escalate_to_engineer",
    "description": "Escalates the issue to a human engineer. Use this when automated fixes fail, or when the error logs indicate a complex issue like",
    "parameters": {
      "type": "object",
      "properties": {
        "summary": { "type": "string", "description": "A brief summary of the findings (health status, log errors) and why you are escalating." }
      },
      "required": ["summary"]
    }
  }
}
```

- Schema guiding LLM on escalation triggers (complex failures) and requiring contextual summary for effective human handoff.

TASK 5: To Append the result to messages

- Properly format and append tool execution results to the conversation history so the LLM can process outcomes.

```
messages.append({  
    "role": "tool",  
    "tool_call_id": tool_call.id, # CRITICAL: Links output to the request  
    "name": func_name,  
    "content": tool_output  
})
```

- Critical step to close the ReAct loop. tool_call_id links result to original request so LLM can correctly interpret outcomes and decide next action.

**Let's go-ahead and publish this Assignment to
our GitHub portfolio.**

System Behavior Demonstration

How the Complete System Works:

1. Investigation Phase:

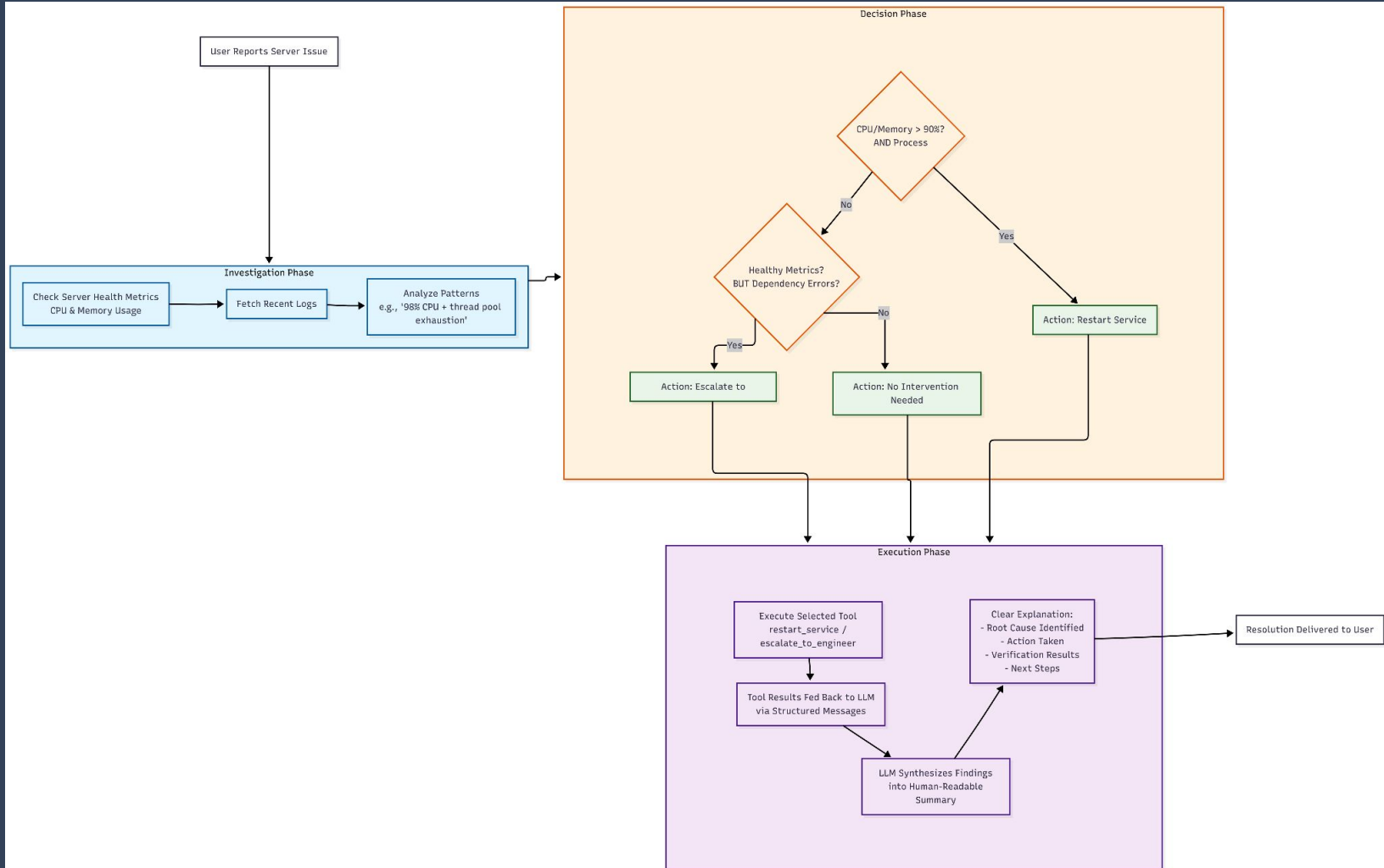
- LLM checks health metrics → fetches relevant logs
- Analyzes patterns (e.g., "98% CPU + thread pool exhaustion" = restart needed)

2. Decision Phase:

- High CPU/memory + process errors → `restart_service`
- Healthy metrics + dependency errors → `escalate_to_engineer`
- Normal metrics + clean logs → no action needed

3. Execution Phase:

- Tool results fed back to LLM via properly formatted messages
- LLM synthesizes findings into human-readable summary
- Clear explanation of actions taken and next steps



What Makes This Solution Production-Ready

Robust Tool Integration:

- Preserves `tool_call_id` for accurate response linking
- Idempotent tools safe for repeated execution

Intelligent Decision Logic:

- Correctly triggers restarts for CPU >90%
- Escalates complex errors (e.g., payment gateway failures) to humans
- Avoids unnecessary actions on healthy systems

What Makes This Solution Production-Ready

Observability:

- Clear execution logging at each step (-> TOOL: ...)
- Structured JSON outputs for reliable parsing
- Full context maintained across tool-call cycles

Resilience:

- Graceful handling of missing servers/metrics
- Consistent error formatting prevents LLM confusion
- Human-in-the-loop escalation path for edge cases

Best Practices

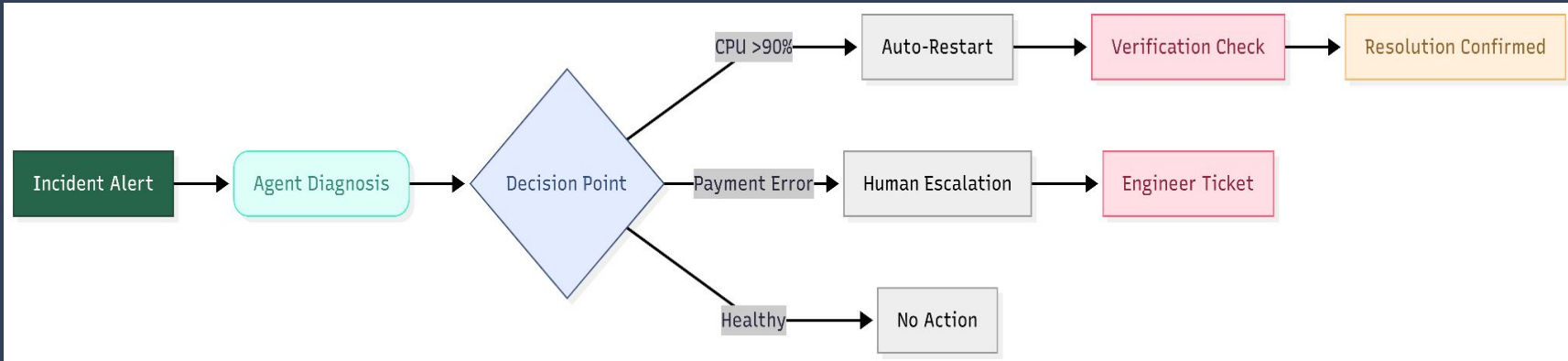
Key Best Practices Demonstrated

- **Tool design:** Each tool does one thing well with clear contracts
- **Error handling:** Graceful degradation when servers aren't found
- **Schema precision:** Parameter descriptions guide LLM usage
- **Conversation management:** Proper tool_call_id matching prevents state corruption
- **Test-driven scenarios:** Realistic test cases validate decision logic

Real-World Applications & Business Impact

From This Assignment to Production Systems

- Add authentication/authorization to tool execution
- Implement circuit breakers for repeated failures
- Add human-in-the-loop approval gates for critical actions
- Integrate with existing monitoring/alerting systems (PagerDuty, OpsGenie)
- Implement comprehensive audit logging for compliance



Real-World Applications & Business Impact

Immediate IT Operations Value

- **MTTR Reduction:** Automated diagnosis cuts mean-time-to-resolution by 60-80% for common issues
- **Tier-1 Automation:** Handles 40-60% of routine incidents without human intervention
- **24/7 Coverage:** Continuous monitoring and response outside business hours

Conclusion & Key Takeaway

- Function calling transforms LLMs from chat interfaces into action-oriented agents – but success depends on:
 - Precise tool definitions with clear contracts
 - Well-crafted system prompts encoding business logic
 - Robust execution loops managing state across interactions
 - Realistic testing with scenario-based validation

This pattern extends far beyond IT support – it's the foundation for any autonomous agent that needs to interact with real-world systems safely and effectively.

